

Indian Institute of Science Education and Research Bhopal

Course: **Data Structures and Algorithms**
Lab Practice Questions

Course Code: **ECS 202**
Date: **07-02-2026**

Programming Guidelines:

- Write a program that allows the user to select an operation using choices.
 - After each selection choice, the program should read the necessary details in the same order as the test case inputs.
 - Print informative messages about the operation result.
 - Use appropriate data structures as specified in each question part.
 - Include comments explaining your approach and choice of data structures.
 - Implement proper exception handling for boundary conditions (full/empty/invalid).
 - Ensure memory management (no leaks, proper destructors, copy constructors).
-

Question 1: Smart University Campus Management System

Context

Design a comprehensive system for IISER Bhopal campus that integrates **multiple data structures** to manage students, courses, library, and cafeteria operations efficiently.

Part A: Student Enrollment & Course Registration

Implement a student database using **Hash Table with Separate Chaining**:

- Store student records mapping `StudentID` $\rightarrow$ `StudentInfo` (name, department, year)
- **Hash function:** Use polynomial accumulation on student ID digits:

$$h(k) = (k[0] + k[1] \cdot 31 + k[2] \cdot 31^2 + \dots) \mod m$$

- Handle collisions using **singly linked lists**
- Implement: `insertStudent()`, `searchStudent()`, `deleteStudent()`
- **Rehash automatically** when load factor $\alpha > 0.75$ (double table size)

Part B: Course Waitlist Management

Manage course registrations using **Circular Queue** :

- Fixed capacity based on maximum course size
- Use **modulo arithmetic** for wrap-around: `rear = (rear + 1) % capacity`
- Fields: `front`, `rear`, `count`, `capacity`
- Operations: `enqueueWaitlist(plotID)`, `dequeueWaitlist()`, `isFull()`, `isEmpty()`
- When seat available, dequeue from waitlist and enroll student
- Exception handling for queue full/empty conditions

Part C: Library Book Checkout with Undo

Implement checkout tracking using **Stack** :

- Each checkout pushes transaction `{bookID, studentID, timestamp}` onto stack
- `undoLastCheckout()`: Pop transaction and reverse the operation
- `displayCurrentBorrowed()`: Show books currently checked out (stack top view)
- Implement using **array-based stack** with exception handling for underflow
- Real-world: Undo accidental checkouts or returns

Part D: Cafeteria Order Processing

Manage food orders using **Deque with Sentinel Nodes**:

- **Doubly linked list** implementation with dummy header and trailer nodes
- `addLast()`: Regular student orders added at rear (FIFO)

- `addFirst()`: VIP/Professor orders added at front (priority)
- `removeFirst()`: Process orders from front
- Implement C++ iterator for traversing pending orders

Integration Requirement: Combine all four parts into one program that allows the user to select the desired operation, where:

- Enrolled students (Part A) can join course waitlists (Part B)
- Students can checkout books (Part C) and place cafeteria orders (Part D)
- Use **template/generic classes** for reusability

Question 2: Real-Time Traffic Management System

Context

A smart city traffic control system integrating **hash tables, priority queues, linked lists, and multiple stacks** for vehicle tracking, route optimization, and emergency handling.

Part A: Vehicle Registry with Universal Hashing

Fast vehicle lookup using **Advanced Hash Table**:

- Map vehicle number plate $\rightarrow$ Vehicle details (owner, type, registration date)
- **Hash code map**: Polynomial accumulation on ASCII values of characters
- **Compression**: MAD (Multiply-Add-Divide) method:

$$h(k) = |a \cdot k + b| \mod m$$

- **Collision resolution**: Separate chaining with **sorted linked lists**
- **Universal hashing**: Randomly select hash function from family H
- Monitor load factor $\alpha = n/m$, **rehash** when $\alpha > 0.7$
- Operations: `registerVehicle()`, `searchVehicle()`, `deregisterVehicle()`

Part B: Traffic Signal Priority Queue

Intersection management using **Priority Queue** (array-based heap or sorted sequence):

- Priority = arrival time for normal vehicles, priority = 0 for emergency vehicles
- Operations: `enqueueVehicle()`, `dequeueHighestPriority()`, `peekNext()`
- **Four separate queues** for 4-way intersection (North, South, East, West)
- Use **Vector ADT** with dynamic resizing for each queue
- Implement sorting by priority using insertion sort or maintain heap property
- Simulate: emergency vehicle arrival triggers immediate green signal

Part C: Route Navigation with Backtracking

GPS navigation history using **Two Stacks** :

- **Stack A (Forward)**: Current route from start to present location
- **Stack B (Backward)**: History for backtracking (undo moves)
- Operations:
 - `goForward(location)`: Push onto A
 - `goBack()`: Pop from A, push onto B
 - `showCurrentRoute()`: Display A from bottom to top
 - `goForwardAfterBack(newLocation)`: Clear B, push onto A
- Real-world: Browser-style back/forward navigation for routes

Part D: Accident Hotspot Tracking

Location-based analysis using **Doubly Linked List with Position ADT**:

- Store accident records: {GPS coordinates, timestamp, severity}
- **Doubly linked list** with `prev` and `next` pointers
- **Position ADT**: Abstract notion of “place” in the list
- Operations:
 - `addAccident()`: Insert at position (chronological order)
 - `removeOldAccidents(days)`: Remove nodes older than threshold

– `getAccidentsInRange(minLat, maxLat, minLong, maxLong)`: Query by GPS bounds

- **Iterator pattern**: Traverse all accidents using `hasNext()`, `next()`
- Sort accidents by frequency using **insertion sort on linked list**

Integration Requirement: Unified traffic system where:

- Registered vehicles (Part A) enter intersection queues (Part B)
 - Routes tracked (Part C) help identify accident hotspots (Part D)
 - Emergency vehicles in queue trigger route recalculation
-

Question 3: Online Auction Platform

Context

An eBay-like auction system managing users, bids, and categories using **hash tables, dynamic arrays, stacks, and n-ary trees** with diverse implementation techniques.

Part A: User Database with Multiplication Hashing

Secure user authentication using **Hash Table** with advanced techniques :

- Map username $\rightarrow$ UserProfile (password hash, email, rating, join date)
- **Hash code**: Horner's rule on username string characters
- **Compression** (Multiplication method):

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor \text{ where } A = \frac{\sqrt{5} - 1}{2} \approx 0.618$$

- This is **Fibonacci hashing** - Knuth's recommended approach
- Operations: `registerUser()`, `authenticateUser()`, `updateRating()`
- Handle collisions with chaining; maintain sorted chains for faster search

Part B: Active Auctions Management

Efficient auction storage using **Vector (Dynamic Array)** with Binary Search:

- Store Auction objects: {itemID, itemName, currentBid, highestBidder, endTime}
-

- **Maintain sorted by itemID** for $O(\log n)$ search
- Operations:
 - `addAuction()`: Insert maintaining sorted order (or sort after batch insert)
 - `findAuctionByID()`: **Binary search** implementation
 - `removeAuction()`: Remove and compact array
 - `updateBid()`: Find auction, update if new bid higher
- **Dynamic resizing**: When full, create new array of size $2N$, copy elements
- Show that add operation is amortized $O(1)$, search is $O(\log n)$

Part C: Bid History and Validation [2.5 Marks]

Track bidding history using **Stack** with auxiliary operations (Lec2):

- Each bid: Push `{bidderID, bidAmount, timestamp}` onto stack
- Operations:
 - `placeBid()`: Validate bid (higher than previous), then push
 - `getLastBid()`: Peek at top without removal
 - `undoBid()`: Pop last bid (admin/correction feature)
 - `isBidValid(amount)`: Check against stack top and reserve price
- When auction ends: Pop all bids to determine winner (highest unique bid)
- Use **array-based stack** with growable capacity if needed
- Implement **exception handling** for empty stack operations

Part D: Category Hierarchy with N-ary Trees

Browse items by category using **Tree with Unbounded Branching**:

- Root: “All Categories”
- Children: Electronics, Fashion, Home, Sports, etc. (arbitrary number per node)
- Each category can have subcategories (e.g., Electronics $\rightarrow$ Phones $\rightarrow$ Smartphones)
- **First-child/Next-sibling representation** (Lec6):
 - Each node has: `firstChild` (points to leftmost child)

- `nextSibling` (points to right sibling)
 - This converts n-ary tree to binary tree representation
- Operations: `addCategory()`, `addSubcategory()`, `listItemsInCategory()`
- **Preorder traversal:** Display full category hierarchy (root before children)
- Count total categories, find height of category tree

Integration Requirement: Complete auction system where:

- Authenticated users (Part A) create auctions in categories (Part D)
 - Active auctions stored in searchable vector (Part B)
 - Bidding history tracked with undo capability (Part C)
-